

docs\CLOSEOUT-r3.md

FAMILY FOTO

===== Sprint Closeout & Release Tag Checklist =====

Release: 0 Major 6 Minor 0 Patch ☒ DEVType: [] alpha [] beta ☒ dev ☒ public ☒ PUBDate/Time Begin: AUG 25, 2025 09:00Released by: MIKE M2025
MIKE M

NOTE. You should begin this checklist in the main, clean

1. Final Code Commit ☒ Create branch, 'git checkout -b sprint/0-4-0-dev' ☒ Ensure all feature/bugfix branches are merged to main

☒ Clear Problems [Show Warnings] [Show Errors] in IDE:☒ Address runtime or type-checking errors☒ Review and resolve compiler warnings:☒ Search "Show Infos" TODO, FIXME, HACK comments:☒ Convert critical items into GitHub issues☒ Add // TODO (backlog) to non-sprint TODOs☒ Remove stale or irrelevant TODOs

2. Final release dashboard

☒ Label open issues [CARRYFORWARD] and move to later milestone ☒ Generate final sprint dashboard, 'python ./tools/dashboard.py --milestone v0.04.0 --save' ☒ Copy to 'docs/release/v0.4.0/dashboard-v0.4.0.txt'

3. Run Final Maven Tests and Build

[] ./tools/run-tests.sh - mvn clean test [] Review output test-results - latest applicable test

[] Append 'tag v0.4.0' to the test folder [] From space folder, mvn clean install

[] Verify final .jar and assets in target/

[] Copy final .jar folder to test desktop ☒ Run .jar from terminal, 'java -jar .\Orbis-0.4.0.jar' ☒ Runverification test case ☒ Verify no runtime exceptions ☒ Verify logs3B ☒ Run Py Tests (PYTHON) → Save Report

4. Create Temporary Tag for Release Summary

[] Run: git tag -a v0.4.0 -m "Draft release v0.4.0" (DO NOT push)

5. Generate Release Summary

[] Run: ./release-summary.sh v0.3.0 v0.4.0

[] Copy output and update:

[] CHANGELOG.md

[] README.md (if version/date listed)

[] Any version metadata files (BuildInfo, VERSIONS.csv, etc.)

FIRST RELEASE

6. Delete Temporary Tag

☒ Run: git tag -d v0.4.0

===== PAGE BREAK =====

7. Generate Final Documents

- ☒ Review and update README.md with:
 - ☒ New features added
 - ☒ Any updated dependencies
 - ☒ Notable instructions for usage or new tools
- ☒ Update CHANGELOG.md with summary of sprint deliverables
- ☒ Confirm USAGE.md, VERSIONS.csv, and milestone files reflect final state

8. Commit Final Documents

— DIRECT MAIN

- ☒ Run: `git add CHANGELOG.md README.md`
- ☒ Run: `git commit -m "Update changelog and docs for v0.4.0 release"`

9. Create Final Release Tag

- ☒ Run: `git tag -a v0.4.0 -m "Release v0.4.0"`
- ☒ Run: `git push origin v0.4.0`

`git tag -d v0.4.0` # delete local tag `git push origin :refs/tags/v0.4.0` # delete remote tag `git tag -a v0.4.0 -m "Release v0.4.0"` # create new tag `git push origin v0.4.0` # push new tag `git log v0.3.99..v0.4.0 > docs/releases/v0.04.0/GIT-LOG-v0.4.0.txt`

10. Create Sprint Artifacts

- ☐ Run or update scripts in `scripts/`:
 - ☐ `scripts/create-chat-bundle-zip.py`
 - ☐ `scripts/github_dashboard.py`
- ☐ Save bundle as `chat-bundles/sprint-vX.Y.Z.zip`
- ☐ Create `docs/releases/sprint-vX.Y.Z.md` with:
 - ☐ Tag
 - ☐ Date
 - ☐ Summary
 - ☐ Contributors
 - ☐ Key commits / PRs

11. Close GitHub Milestone

- ☒ Review milestone issues:
- ☒ Mark completed
- ☒ Label uncompleted as `carry-forward`
- ☒ Close milestone `vX.Y.Z`

12. Start Next Sprint Prep

- ☒ Create new milestone `vX.Y+1.0`
- ☒ Optionally create branch `sprint/x-y-z-dev`
- ☒ Groom backlog and select next sprint stories

13. Notify Team / Stakeholders

- ☒ Share release summary and tag reference
- ☒ Confirm milestone closed on GitHub
- ☒ (Optional) Draft release notes on GitHub using CHANGELOG.md

Date/Time Completed:

Aug 125 / 2025 13:43

##=====

DEV MNK MNK

AUG 25, 2025

⊕ SPRINT RELEASE V0.6.0

- ORBIS 2D
- FAMILY FOTO

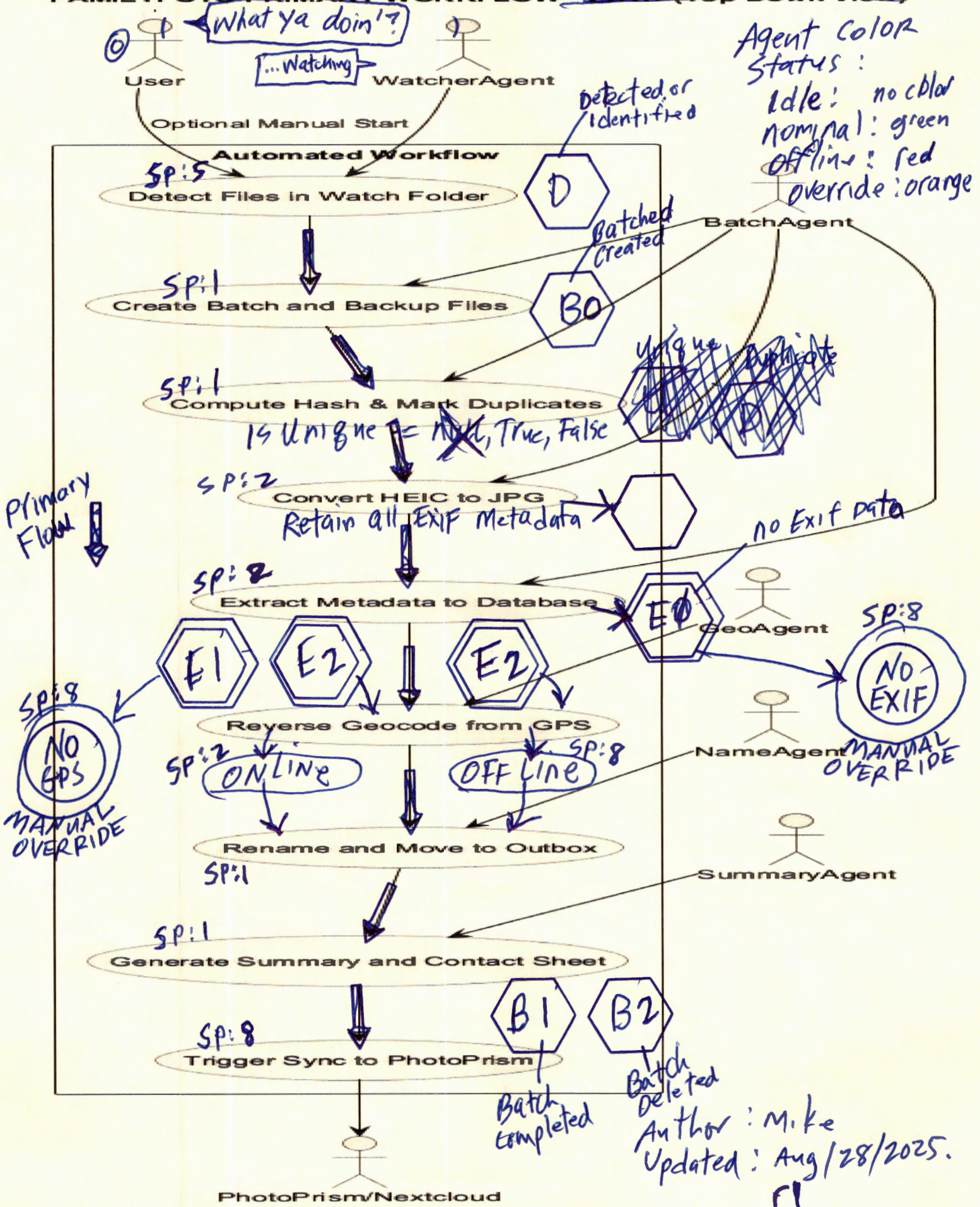
⊕ FAMILY FOTO V0.6.0

- foto namer
- foto mover
- foto menu
- clean inbox Empty
- clean inbox Junk
- README
- CHANGE LOG
- USAGE
- ~~PY~~ DOCS
- UML DOCS
- USER GUIDE
- UTILITY - Location / Geo
- UTILITY - File
- UTILITY - Config
- UTILITY - Data
- photo-config → Foto.config
- Logger
- - Report &
- ☒ Setup Repo milestones / Dashboard
- SSH Keys
- ☒ Dashboard + ENV
- - Make it a game
- - foto tagger
- - folder tagger
- SQLITE3 Photos.db.

V07.0

Rapid Flashings vs. slow Flashing vs Solid

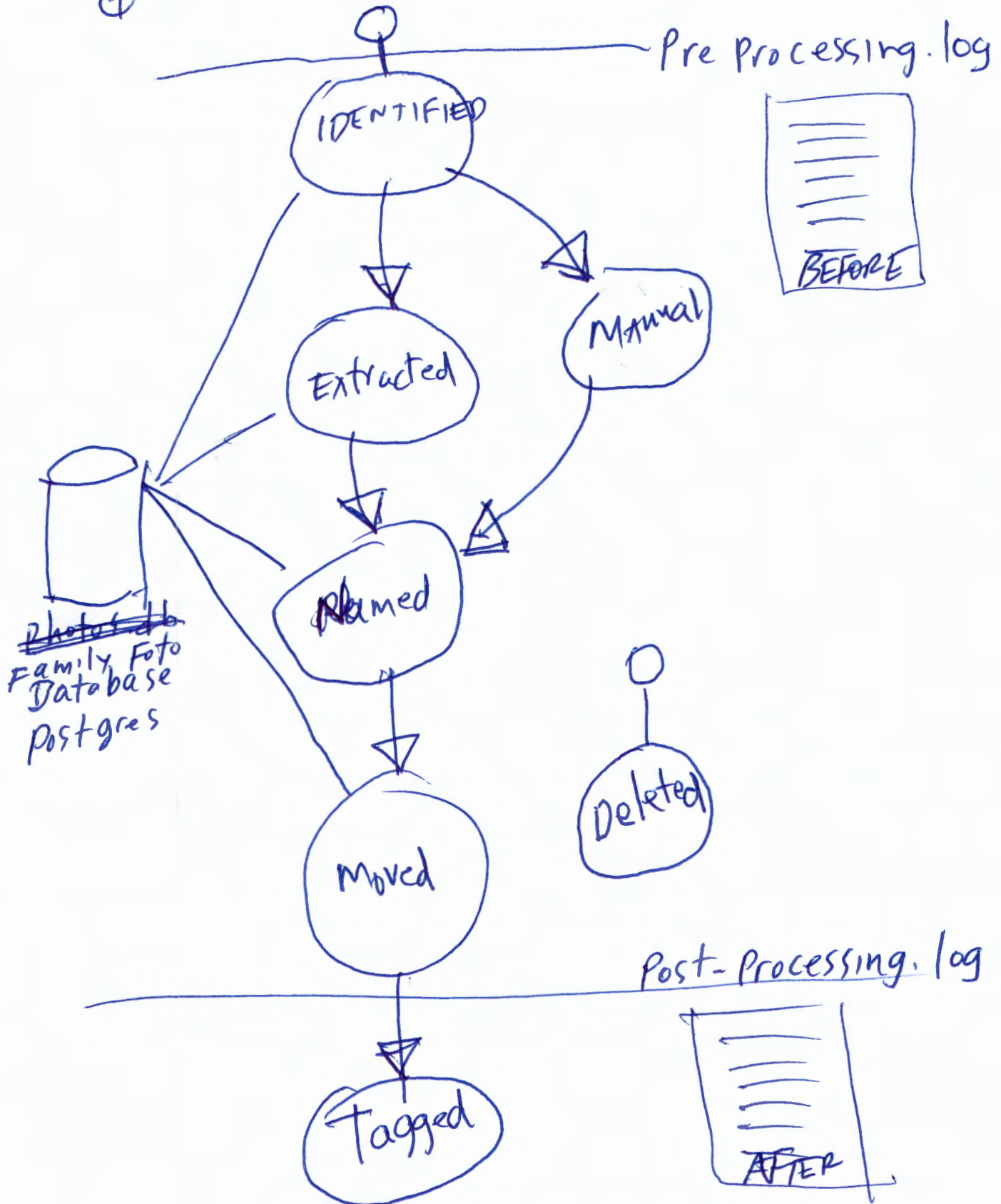
FAMILY PHOTO PRIMARY WORKFLOW - v1.11 (Top Down View)



FAMILY FOTO

AUG 26, 2025

④ STATUS STATE DIAGRAM



V0.7.0

FAMILY FOTO
AUG 29, 2025

user override

WATCHER STATE

Idle

Nominal

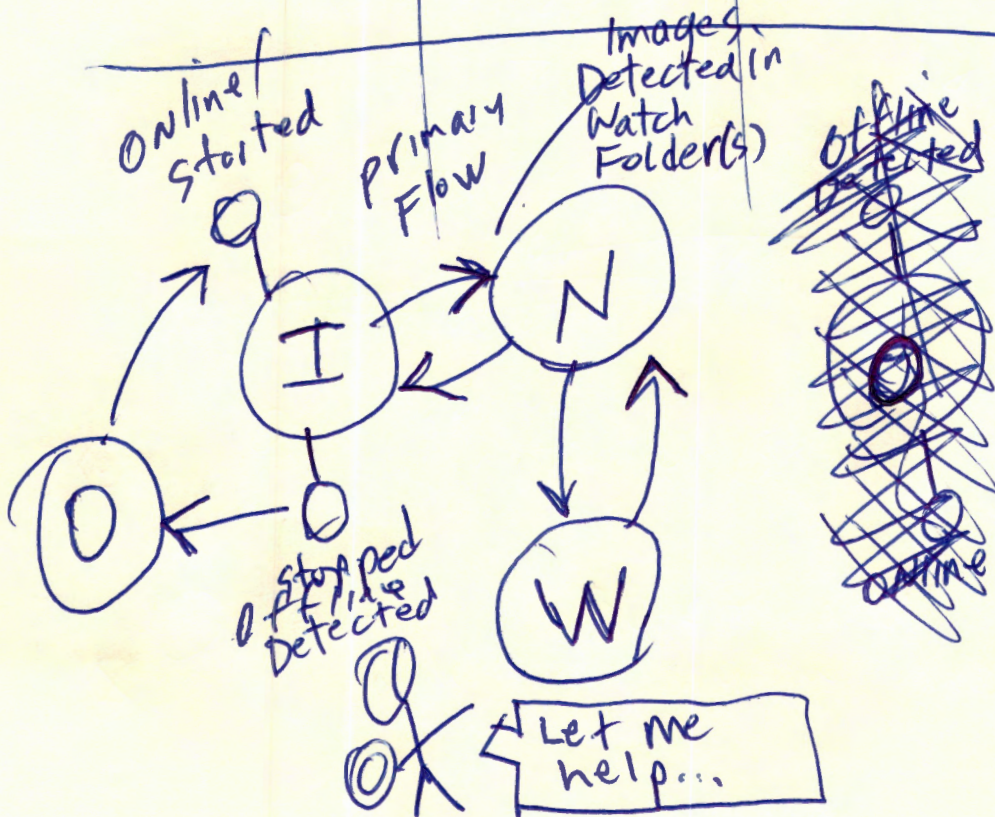
Offline

Waiting
override
Manual
required.

rapid

slow

solid

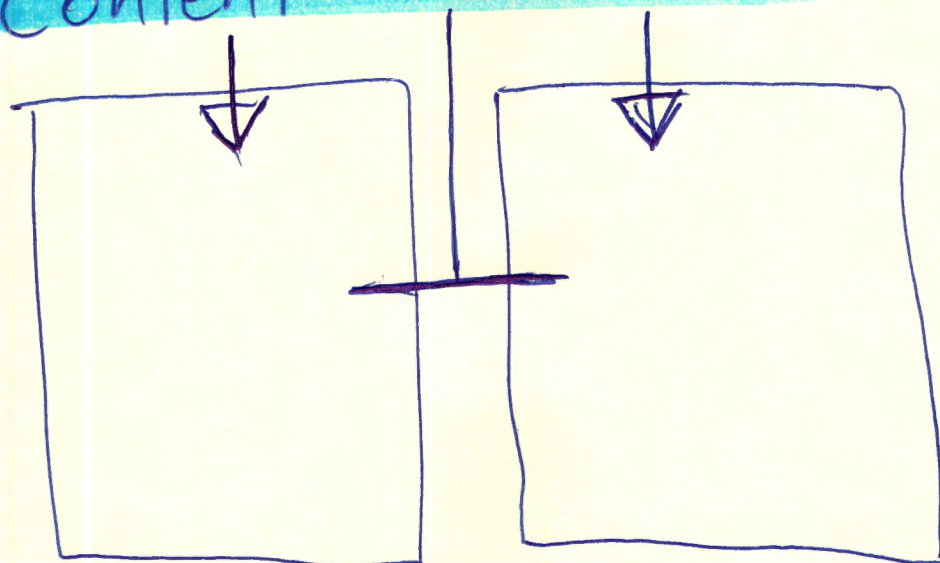


FAMILY FOTO
AUG 28, 2025

⑥ ⊕ IF AVAILABLE COLLECT ALL
EXIF DATA
see EXIF-TAGS.md (r1)

⑥ ⊕ APPLE PAIRED LIVE.MOV
— Live Photo Video Index
— Pair .mov w/ Photo

Content Identifier



Ocad. HEIC

Ocad. mov

~~#12580096.~~

⑥ ⊕ Need pyheif to get
all HEIC EXIF Data

⊗ ⊕ need during conversion
visual studio Build Tools

FAMILY FOTO

AUG 28, 2025

⊕ Foto Batch Break up
INBOX FOLDERS INTO
AV GROUP OF 500 FOTOS
RELATED — WILL BE ABLE
TO COMPLETE PIPELINE
IN LESS THAN 5-10 MINUTES

⊕ batch table

⊕ FOTO BACKUP + ZIP

USE TREE AND ZIP INVENTORY
TO BACKUP THE BATCH
PRIOR TO STARTING WORKFLOW

⊕ BATCH METRICS

- START
- STOP
- COUNT START
- COUNT END
- GPS CENTER
- APPROX. DATETIME
- MOST LIKELY CAMERA

⊕ camera table

- From
- TO
- owner
- unique camera ID.

FAMILY FOTO

AUG 28, 2025

⊕ FOTO DASH

⊕ FOTO CAM

⊕ POSTGRES VS SQLITE
FOR END USER MAYBE
GIVE THEM OPTIONS.

IF NO EXIF
USER CAN TAG
THE BATCH WITH
A FOTO ~~PLACE~~

POSTGRES	SQLITE	CSV
----------	--------	-----

⊕ FOTO **PLACES** — LIST OF MY HOUSES

⊕ FOTO BUG / GIT PROJECTS

⊕ FOTO HELP

⊕ FOTO JAR — JAVA

⊕ FOTO ADMIN 4 — TOMCAT

⊕ FOTO DOCKER

⊕ FOTO FLOW WORKFLOW

WEB MONITOR
STATUS — NEW FOTOS DETECTED
CREATE BATCH.

FROM
TO
CAMERAS
PEOPLE

FAMILY FOTO

AUG 28, 2025

⊕ A BATCH JOB IS AUTO
MATICALLY CREATED ANY
TIME INBOX HAS FOLDERS

— Agent manager
monitors INBOX

— Finds related Fotos

⊙ put them in a group

⊙ Adds the photo to database
HASH

⊙ creates a Batch data
record

⊙ Backup with
Contact sheet
zip Folder
zip inventory.

⊙ marks Batch READY
By ~~putting~~ appending a
small text file.

⊙ Family Foto is Run — Looks
For new Batch Ready.

⊕ FAMILY DOC — similar to
Family Foto but w/ .PDF .DOC

docs\WORKFLOW-r3.md

FAMILY PHOTO WORKFLOW (BatchAgent Enhanced)

Author: Mike Mattinson

Updated: Sep/29/2025 – r3

AUTOMATED WORKFLOW STEPS

-  STEP 1 - Load config and setup folders
-  STEP 2 - Auto-initialize PostgreSQL database if missing
-  STEP 3 - Detect files in watch folder and form batches
-  STEP 4 - Batch initialization:
 - Backup original files
 - Compute SHA256 and UUID
 - Identify duplicates and mark them in DB
 - Retain only 1 primary photo per hash
 - Convert .HEIC → .JPG (preserve EXIF)
 - Rename files in-place (initial UUID or timestamp-based)
 - Create batch.meta.json with status
-  STEP 5 - Evaluate batch metadata quality
-  STEP 6 - Extract EXIF metadata:
 - DateTimeOriginal
 - CameraModel
 - GPS coordinates
 - Store to DB
-  STEP 7 - Reverse geocode (if GPS available):
 - Online via Nominatim API or other
 - Offline via local shapefile/CSV
 - Lookup city_id from DB
-  STEP 8 - Rename and move photos:
 - Use geocode + timestamp to rename: YY-MM-DD_GEOCODE_HH-MM.jpg
 - Organize into folder: YYYY-MM (City, State)
 - Update status = Finalized
-  STEP 8b - Duplicate handling:
 - If hash exists in DB, mark as DUPLICATE
 - Link duplicate to canonical source

- Skip renaming/moving for dupes
- ✓ STEP 9 - Cleanup:
 - Remove empty folders in inbox/watch
 - Optionally archive source files
 - Queue thumbnail sync to PhotoPrism or Nextcloud
- ✓ STEP 10 - Generate summary:
 - summary.csv with camera usage, year, file type
 - contact_sheet.jpg per output folder

! SEMI-AUTOMATED & USER-ASSISTED PATHS

- ! STEP 5 - If Date but no GPS:
 - Try folder name or config-based fallback
 - Prompt for manual geotag (CLI or GUI dashboard)
- ! STEP 6/7 - If Date missing or malformed:
 - Mark file as status = Missing
 - Defer until human provides EXIF override or metadata patch
- ⊖ STEP 7 - If GPS also missing:
 - Flag for human review
 - Prompt via future dashboard or batch-level TODO

🗨 FUTURE UPGRADE IDEAS

- ☐ Predict location based on date & camera model history
- ☐ Learn camera usage patterns per user/device
- ☐ Visual dashboard to show photo clusters on map
- ☐ GUI prompt or approval panel for flagged photos
- ☐ Smart camera alias learning from repeated patterns
- ☐ Location inference using batch group clustering
- ☐ Embed thumbnail and metadata into EXIF comments or IPTC
- ☐ Add QR code to images linking to DB record (for prints)
- ☐ Editable contact sheet metadata (who/what/where)
- ☐ Photo timeline view with gaps and groupings
- ☐ GUI dashboard with status breakdown:
 - ✓ Finalized
 - ! Pending GPS
 - ⊖ No timestamp
 - ⌛ Needs user override
- ☐ Auto-test batches with edge cases and sample images
- ☐ Plugin support for custom pre-processing/post-processing

docs\AGENTS-r1.md

FAMILYFOTO AGENTS – Modular Architecture (r1)

Author: Mike Mattinson
Updated: Sep/29/2025 – r1

OVERVIEW

To support scalable, testable, and maintainable automation, the FamilyFoto pipeline is split into modular agents. Each agent is responsible for a distinct phase of the workflow and updates shared resources such as the database and `batch.meta.json`.

AGENT ROLES

WatcherAgent

+ *Watcher Monitor*

- Monitors the `WATCH_FOLDER` for new photo files
- Detects new content or `.trigger` file as batching signal
- Optionally logs photo counts or readiness
- (Future) May notify downstream agents or flag DB
- Does NOT move, group, or rename files
- Leaves batch grouping to BatchAgent
- Acts as the “eyes” of the system – lightweight, event-driven

BatchAgent

- Activated manually or downstream from WatcherAgent
- Backs up files to `BACKUP_DIR`
- Moves files into an `INBOX_DIR/BATCH-*` folder
- Creates a `batch.meta.json` with initial metadata
- Computes SHA256 hash + UUID for each photo
- Converts `.HEIC` to `.JPG` (retaining EXIF)
- Extracts EXIF metadata (Date, GPS, CameraModel)
- Flags duplicates and stores results in the database
- Marks batch as `READY` for geocoding and naming
- Performs all heavy lifting — batching, hashing, metadata extraction, and backup

GeoAgent

- If GPS is present, performs reverse geocoding:
 - Online (e.g. Nominatim API)
 - Offline (e.g. OSM CSV or shapefiles)
- If GPS is missing:

- Attempts location from folder context or default fallback
- Flags file for manual override
- Updates city_id, state, country in the DB

● NameAgent

- Builds short GEOCODE ID (e.g. UT-SLC)
- Renames files using: YY-MM-DD_GEOCODE_HH-MM.jpg
- Moves photos to output folders: YYYY-MM (City, State)
- Updates status to Finalized

● SummaryAgent

- Generates contact sheet(s) for each output folder
- Exports CSV summary of batch contents:
 - File types, date ranges, camera usage
- Flags anomalies (missing EXIF, duplicates, etc.)

Area	WatcherAgent ●	BatchAgent ●	Overlap?
Primary Role	Detect new files in watch/ and trigger batching	Process and register batches once file set is identified	✗
Runs as a daemon	☑ Polls regularly or waits for trigger	✗ Usually invoked per batch (script or CLI)	✗
Target folder	WATCH_FOLDER	INBOX_DIR and BACKUP_DIR	🔄 transition
Trigger mechanism	☑ Detects new files or .trigger file	✗ Assumes files already moved into inbox	✗
Batch file grouping	🚧 Possible future feature (early filtering)	☑ Groups files into batches (300–500), tags them with metadata	⚠ Some
Backup of source files	✗ Not responsible	☑ Copies to BACKUP_DIR	✗
Hash + UUID generation	✗	☑ Computes SHA256 and UUID for each photo	✗
EXIF extraction	✗	☑ Reads EXIF and evaluates camera/date/GPS info	✗
Batch folder creation	✗	☑ Creates BATCH-YYYY-MM-DD_xxxx folder in inbox	✗
batch.meta.json generation	✗	☑ Writes initial batch metadata	✗
Duplicate detection	✗	☑ Checks for duplicates using hash	✗
Logging activity	☑ Logs detection events	☑ Logs batch creation steps	☑
Triggers next agent	🚧 Planned (e.g. triggers BatchAgent or flags DB)	✗ Completes and passes to GeoAgent	✗

WATCHER VS BATCH OVERLAP

Category

Summary

Category	Summary
Responsibility	Watcher = detection, Batch = processing
Temporal Scope	Watcher is continuous, Batch is discrete
Overlap	Minimal – only in logging and possibly in future early file filtering
Interface Point	Watcher <i>hands off</i> files to BatchAgent via file move or trigger

STATUS PIPELINE

Each agent transitions the batch through statuses:

- batch.status = Initialized
 - watcher detects and groups photos
- batch.status = Registered
 - batch hashes, backups, metadata extracted
- batch.status = Located
 - reverse geocode complete, city/state/country resolved
- batch.status = Named
 - renamed and organized into final folders
- batch.status = Finalized
 - contact sheets created, summary complete

PROPOSED FILE STRUCTURE

- familyfoto/
 - agents/
 - watcher_agent.py # Detects new files and creates batches
 - batch_agent.py # Handles backup, hashing, EXIF extraction, conversion
 - geo_agent.py # Reverse geocoding using GPS or fallback
 - name_agent.py # Renames photos and moves to structured folders
 - summary_agent.py # Generates contact sheets and summary reports

Each file implements a CLI entrypoint or callable class with full logging and config support.

FUTURE: Master Controller

A BatchRunner or fffcli.py script can invoke each agent in order, or allow flags like:

```
fffcli.py run-batch --batch-id B20250929 --from watcher --to summary
```

CORE AGENT LAYERS

Agent Name	agents/	commands/	logic/	utils/
WatcherAgent	☒ watcher_agent.py	☒ watcher_agent.py	☒ watcher_agent.py	☒ watcher_agent_utils.py
BatchAgent	☒ batch_agent.py	☒ batch_agent.py	☒ batch_agent.py	☒ batch_agent_utils.py
GeoAgent	☒ geo_agent.py	☒ geo_agent.py	☒ geo_agent.py	☒ geo_agent_utils.py
NameAgent	☒ name_agent.py	☒ name_agent.py	☒ name_agent.py	☒ name_agent_utils.py
SummaryAgent	☒ summary_agent.py	☒ summary_agent.py	☒ summary_agent.py	☒ summary_agent_utils.py

LAYER ROLES

Layer	Role
agents/	Defines high-level class with .run() or .process() method
commands/	CLI interface using argparse, --batch-id, --config, etc.
logic/	Core business logic functions used by the agent class
utils/	Reusable support functions (I/O, folder helpers, filtering, logging)

FAMILY PHOTO

AUG 28, 2025

Big Font

INBOX FOTOS

1. ~~Folder 233-1600~~

2. ~~Folder 2-16~~ 450

3. _____

4. _____

5. _____

6. more/next 3

7. options

Total
Tracked
Photos

total
remain
photos

Select 2

Big Font
FOLDER 2-

16-450

1. Name

2. Find

3. moving

4. manual override

5. options